



网络安全创新创业实践

实验 5、实验 6

基于 Microsoft SEAL 的 BFV 同态卷积实现与旋 转次数优化分析

程洪鑫 202300460051 网安 23.1

2026 年 7 月 20 日

目录

1	实验目的	1
2	实验背景	1
2.1	同态加密理论	1
2.2	BFV 同态加密方案	1
2.3	Microsoft SEAL 开源库	1
3	实验流程	2
3.1	明文卷积	2
3.2	BFV 参数初始化	3
3.3	创建密钥和计算对象	3
3.4	数据编码与加密	4
3.5	密文卷积	4
3.6	Mask 提取有效输出	5
3.7	解密密文卷积结果	5
4	实验结果	6
5	旋转次数优化分析	6
5.1	打包、旋转与累加策略	6
5.2	当前实现的旋转次数统计	6
5.3	理论最小旋转次数分析	7
6	总结	7

1 实验目的

本实验旨在学习全同态加密的基本原理及其应用，选用 Microsoft SEAL 开源全同态加密库中的 BFV 方案，实现单输入单输出的 4×4 输入与 3×3 卷积核（步长 1，无填充）的密文卷积运算，并验证解密后的卷积结果与明文卷积结果的一致性。在此基础上，采用“打包 $\rightarrow$ 旋转 $\rightarrow$ 累加”策略，对密文卷积中的旋转操作进行分析，探究当前实现的旋转次数是否达到理论最小值，从而理解同态卷积的实现原理及其性能优化方法。

2 实验背景

2.1 同态加密理论

传统加密算法（如 AES、SM4 等）能够保证数据的机密性，但加密后的数据无法直接进行计算。若需要对数据进行处理，则必须先解密，再完成计算，最后重新加密。这种方式不仅增加了计算开销，而且在解密过程中会暴露原始数据，难以满足云计算、隐私计算等场景的数据安全需求。

同态加密是一类特殊的加密技术，其核心思想是在密文状态下直接完成运算，计算完成后再统一解密，得到的结果与直接对明文进行计算的结果一致，即满足

$$Dec(f(Enc(m))) = f(m)$$

其中， $Enc(\cdot)$ 和 $Dec(\cdot)$ 分别表示加密与解密操作， $f(\cdot)$ 表示需要执行的计算。

卷积运算本质上由乘法和加法组成，因此只要同态加密方案能够支持加法同态和乘法同态，就能够直接在密文状态下完成卷积计算，而无需暴露原始数据，从而保证数据隐私。

2.2 BFV 同态加密方案

本实验采用 Microsoft SEAL 提供的 BFV 同态加密方案。BFV 是一种面向整数计算的全同态加密方案，支持加法同态和乘法同态，即

$$Enc(a) + Enc(b) = Enc(a + b)$$

以及

$$Enc(a) \times Enc(b) = Enc(ab)$$

因此，任何由加法和乘法组成的计算理论上都能够在密文空间中完成。

除此之外，BFV 支持 Batch Encoding（批处理编码）技术，可以将多个整数同时打包到同一个密文中，实现单指令多数据（SIMD）式计算，提高同态运算效率。本实验正是利用这一特性，将整个 4×4 输入矩阵编码到同一个密文中，再结合密文旋转完成卷积计算。

2.3 Microsoft SEAL 开源库

Microsoft SEAL 是开源全同态加密库，支持 BFV、BGV 和 CKKS 等多种同态加密方案，并提供了完善的密钥生成、编码、加密、解密以及密文计算接口，是目前应用最广泛的全同态加密库之一。

本实验主要使用了以下几个核心组件：

- **BatchEncoder:** 负责普通整数与 SEAL 明文对象之间的编码与解码，实现多个数据的批量打包。
- **Encryptor:** 利用公钥将明文加密为密文。
- **Evaluator:** 负责密文上的同态运算，包括密文加法、明文乘法以及密文旋转等操作。
- **Decryptor:** 利用私钥完成密文解密。
- **KeyGenerator:** 负责生成公钥、私钥以及密文旋转所需的密钥。

本实验基于 SEAL 提供的 BFV 方案，实现了密文卷积及其旋转优化分析，从而验证全同态加密在卷积计算中的可行性。

3 实验流程

3.1 明文卷积

首先定义输入矩阵和卷积核，并采用普通二维卷积算法计算明文卷积结果，其输出作为后续验证密文卷积正确性的参考结果。

输入矩阵定义如下：

```
1 vector<vector<int64_t>> input_matrix =
2 {
3     {1,2,3,4},
4     {5,6,7,8},
5     {9,10,11,12},
6     {13,14,15,16}
7 };
```

卷积核定义如下：

```
1 vector<vector<int64_t>> kernel =
2 {
3     {1,0,1},
4     {0,1,0},
5     {1,0,1}
6 };
```

由于输入尺寸为 4×4 ，卷积核尺寸为 3×3 ，步长为 1 且不进行 Padding，因此输出尺寸为

$$(4 - 3 + 1) \times (4 - 3 + 1) = 2 \times 2.$$

程序利用四层循环完成二维卷积计算：

```

1  for(int i = 0; i < 2; i++)
2  {
3      for(int j = 0; j < 2; j++)
4      {
5          int64_t sum = 0;
6
7          for(int x = 0; x < 3; x++)
8          {
9              for(int y = 0; y < 3; y++)
10             {
11                 sum += input[i+x][j+y] * kernel[x][y];
12             }
13         }
14
15         output[i][j] = sum;
16     }
17 }

```

计算得到的明文卷积结果将作为密文卷积正确性的验证标准。

3.2 BFV 参数初始化

完成明文卷积后，程序开始初始化 BFV 同态加密环境。首先创建加密参数对象，并设置多项式阶数、密文模数以及明文模数：

```

1  EncryptionParameters parms(scheme_type::bfv);
2
3  size_t poly_modulus_degree = 4096;
4
5  parms.set_poly_modulus_degree(poly_modulus_degree);
6  parms.set_coeff_modulus(CoeffModulus::BFVDefault(poly_modulus_degree));
7  parms.set_plain_modulus(PlainModulus::Batching(poly_modulus_degree, 20));
8
9  SEALContext context(parms);

```

3.3 创建密钥和计算对象

随后根据 BFV 参数生成同态加密所需的各种密钥，包括公钥、私钥以及支持密文旋转操作的 Galois Keys：

```

1  KeyGenerator keygen(context);
2
3  SecretKey secret_key = keygen.secret_key();
4

```

```

5 PublicKey public_key;
6 keygen.create_public_key(public_key);
7
8 GaloisKeys galois_keys;
9 keygen.create_galois_keys(galois_keys);

```

之后创建 Encryptor、Evaluator、Decryptor 以及 BatchEncoder 等对象，为后续的加密、解密及密文运算做好准备。

3.4 数据编码与加密

由于 BFV 无法直接处理普通数组，因此首先需要利用 BatchEncoder 将输入矩阵编码为 Plaintext:

```

1 Plaintext plain;
2 encoder.encode(input, plain);

```

随后利用公钥完成加密:

```

1 Ciphertext encrypted;
2 encryptor.encrypt(plain, encrypted);

```

经过该步骤后，输入数据已经转换为 Ciphertext，程序无法直接访问其中的数据，只能通过 Evaluator 提供的同态运算接口完成计算。

3.5 密文卷积

由于密文无法像普通数组一样随机访问，因此实验采用“打包 → 旋转 → 累加”策略完成卷积计算。

对于卷积核

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix},$$

真正参与计算的位置对应的 Slot 偏移量分别为

0, 2, 5, 8, 10.

程序首先定义对应的旋转偏移:

```

1 vector<int> rotations = {0,2,5,8,10};

```

随后遍历所有偏移量。当偏移量为 0 时直接使用原始密文；否则调用

```

1 evaluator.rotate_rows(encrypted, r, galois_keys, temp);

```

完成密文旋转。
每完成一次旋转后，再利用

```
1 evaluator.add_inplace(result, temp);
```

将旋转后的密文累加，实现卷积窗口对应元素的求和。

3.6 Mask 提取有效输出

由于旋转累加后整个密文都会产生计算结果，而本实验真正需要的输出仅为左上角 2×2 区域，因此构造如下 Mask：

```
1 vector<int64_t> mask =  
2 {  
3     1,1,0,0,  
4     1,1,0,0,  
5     0,0,0,0,  
6     0,0,0,0  
7 };
```

随后调用

```
1 evaluator.multiply_plain(  
2     result,  
3     mask_plain,  
4     final_result);
```

保留有效输出位置，其余 Slot 全部置零。

3.7 解密密文卷积结果

完成密文卷积后，首先利用私钥完成解密：

```
1 decryptor.decrypt(final_result, decrypted);
```

解密得到的是 SEAL 内部的 Plaintext，并不能直接读取，因此还需要利用 BatchEncoder 进行解码：

```
1 encoder.decode(decrypted, output);
```

最终恢复普通整数数组，并输出卷积结果。

4 实验结果

```
cheng@LAPTOP-VLSK33D7:~/task5/build$ ./conv
Plain convolution result:
30 35
50 55

Encryption finished

Decrypted convolution result:
30 35
50 55
```

图 1: 卷积结果

可以看出，密文卷积解密后的结果与明文卷积结果完全一致，验证了同态卷积的正确性。

5 旋转次数优化分析

5.1 打包、旋转与累加策略

在普通卷积计算中，可以通过数组下标直接访问卷积窗口内任意位置的数据，例如

$$input[i + x][j + y].$$

然而，在 BFV 同态加密方案中，整个输入矩阵已经利用 BatchEncoder 打包（Packing）到同一个密文中。此时，密文中的数据不能像普通数组一样按照下标随机访问，也不能直接读取某一个 Slot 的值。

为了完成卷积计算，需要利用密文旋转（Rotation）操作，将目标数据移动到相同的 Slot 位置，再利用密文加法完成累加。因此，密文卷积通常采用“打包 → 旋转 → 累加”策略，其基本流程如下：

1. 将整个输入矩阵打包到一个密文中；
2. 对密文执行若干次旋转，使卷积窗口中对应位置的数据移动到相同 Slot；
3. 将所有旋转后的密文进行累加，得到卷积结果。

因此，密文旋转次数直接影响同态卷积的计算复杂度，也是同态卷积优化的重要目标之一。

5.2 当前实现的旋转次数统计

本实验采用的卷积核为

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}.$$

其中，仅有五个位置的元素非零，因此真正参与卷积计算的数据对应的 Slot 偏移量分别为 0, 2, 5, 8, 10.

在代码中，当偏移量为 0 时，并不会调用 `rotate_rows()`，而是直接使用原始密文，因此真正执行旋转操作的仅有 2, 5, 8, 10 四个偏移，对应调用 `rotate_rows()` 共 4 次。

因此，本实验实际旋转次数为 4 次。

5.3 理论最小旋转次数分析

为了判断当前实现是否已经达到最优，需要进一步分析理论上最少需要多少次旋转。

观察卷积核可以发现，其五个非零元素分别对应输入矩阵中的五个不同偏移位置：

$$0, 2, 5, 8, 10.$$

其中，偏移量 0 可以直接利用原始密文，无需任何旋转。

对于其余四个偏移，每一个偏移对应的数据都位于不同的 Slot 中。由于一次密文旋转只能按照固定偏移移动所有 Slot，而不能同时获得多个不同偏移的数据，因此每一个非零偏移至少需要执行一次独立的旋转操作。

因此，理论最少旋转次数为

$$N_{min} = 5 - 1 = 4,$$

其中减去的 1 表示偏移量 0 无需旋转。

若省略任意一次旋转，则对应位置的数据将无法参与卷积累加。

故当前实现已经达到理论最小旋转次数。

6 总结

本实验基于 Microsoft SEAL 开源库，实现了 BFV 同态加密方案下的 4×4 输入与 3×3 卷积核的密文卷积计算，并验证了解密后的卷积结果与明文卷积结果完全一致，证明了同态卷积实现的正确性。

实验过程中，利用 BatchEncoder 将输入矩阵打包到单个密文中，再采用“打包 → 旋转 → 累加”策略完成卷积运算，实现了密文状态下的数据处理。同时，对密文卷积中的旋转操作进行了统计与理论分析，证明当前实现仅需执行 4 次密文旋转，与理论最小旋转次数一致，说明该实现已经达到较优的计算效率。